King Saud University

College of Computer and Information Sciences

Computer Science Department

“Assignment 1: AES-128 Brute Force Attack Simulation”

CSC 489– Homework Report

Prepared by:

KHALED AHMED S ALSHEHRI 444101536

Supervised by:

Dr. ALMUHTADI JALAL FAREED J

Second Semester 1447/2025

CSC 489 Assignment 1: AES-128 Brute Force Attack Simulation

Introduction

This project was implemented using **Python 3** and the **PyCryptodome** library, a self-contained cryptographic package. The simulation utilizes the Advanced Encryption Standard (AES) with a 128-bit key in Electronic Codebook (ECB) mode

Key features:

The key futures of using PyCryptodome lib and python 3 for this project include:

1. **Key Generation:** A random 128-bit (16-byte) key is generated using a cryptographically secure random number generator.
2. **Padding:** Since AES is a block cipher, the **PKCS7** padding scheme was used to ensure the plaintext paragraph aligns with the 16-byte block requirement.
3. **Brute Force ability:** The simulation iterates through all possible hexadecimal combinations until the original plaintext is successfully recovered or a time limit is reached.

Results

The analysis is based on 10 tests done with the same Plaintext, the results are then taken the average of.

Data type	Data value
Plaintext	(Assignment objective): The goal...
Original Key (Hex) **RANDOMLY GENERATED EACH TURN**	EDE9228C5718535F7F0935D45432C17A
Ciphertext (Hex) **CHANGES BASED ON THE RANDOMLY GENERATED KEY**	A10B1168E5112ED62B757E626C9D0AD6BC5 ...

Results based on case:

Case	Missing bits	# of attempts	Time taken	Result
Case 1	**8 bits**	**120 attempts made**	**0.00 Seconds**	**Success**
Case 2	**32 bits**	**159420137 attempts made**	**1789.32 seconds**	**¼ Success** **¾ Time - out**
Case 3	**64 bits**	**141855646 attempts made**	**1800.00 seconds**	**Time - out**

Help provided:

During the development of this simulation, several resources were consulted to understand the implementation of PyCryptodome and AES

PyCryptodome:

Provided the syntax for AES.new() and padding requirements.

Github repos:

GitHub Repository (example Nabeelcn25): I took inspiration from various open-source snippets showing how to convert integers to bytes for key guessing.

Stack overflow:

Stack Overflow search was conducted on handling UnicodeDecodeError when attempting decryption with incorrect keys.

Ai-Gen models:

Provided help debugging when some keys were not working, helped give some functions in the libraries that I did not know of (such as the random key

generator. Fixed some CPU heavy load by offering a return cartridge advice to make the progress look like an update bar.

Security Implications:

Based on these findings, AES-128 remains secure against brute force attacks. If a computer takes 30 minutes to fail at cracking just 32 bits (1/4 the key!), the time required to crack the full 128 bit key would be extremely high. The simulation proves that even partial knowledge of a key does not guarantee a successful breach if the remaining bit count is sufficiently high.

This simulation was tested on a high-level computer in a computer Café, and even there it took a very long time (29 minutes in ¼ of times) to crack the 32 bit key, and never the 64 bit key. This further proves the security of AES-128.

Future Works

As a student having taken courses on what could possibly make cracking the 32 or 64 keys faster, these following works could be helpful.

GPU Multi-threading programs: In courses like parallel programming,

We used CUDA to help execute some programs on GPU instead of CPU

Much faster. This could prove helpful.

Key Guessing: The program provided here worked incrementally,

Random or different pattern for key searching could be helpful.

Acknowledgements and sources:

- **Github repos:** Legrandin, pycryptodome, Sessions_Python
- **Reddit** thread
- **Stack overflow:** Problem with padding, Error handling issue
- **Ai Models:** Helped in the start with how to start, and then along the way when stuck they were asked for guidance and explanations of why

certain functions were behaving in a certain way and what could be done to fix them.